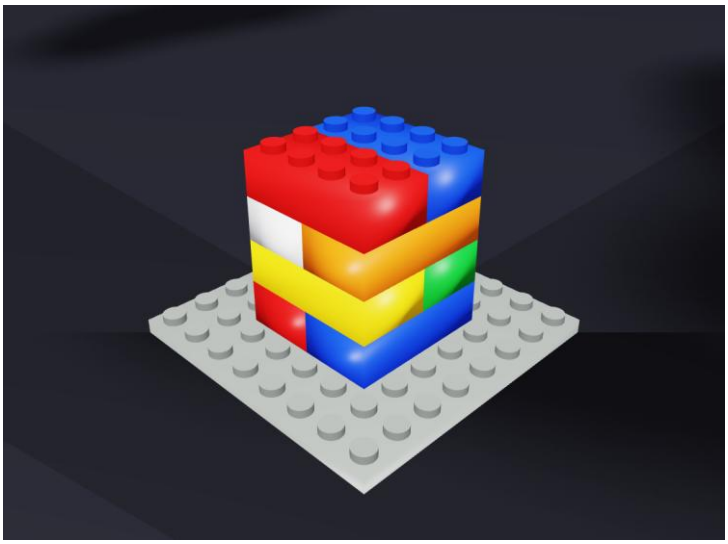
Group 10

DESIGN PROPOSAL

LEGO Robotic Scaffolding Assembly and Wriggle Fit

Dual-arm assembly in which one arm builds and the other acts as temporary scaffolding

---



Team members

1. Tanvir, HOSSAIN - 21390630
2. Sakibul Haque, SAKIB - 21287766
3. Minjian LI - 21112296
4. She Yuhang - 21417311

Course: ISDN 5240: Robotic Fabrication with Python
Date: 27 October 2026
Semester: Fall 2026

Table of Contents

1. Introduction **3**
1.1 Dual-arm LEGO assembly as a scaffolding problem 3
1.2 Objectives 3
2. Methodology **3**
3. Design Specification and Progress **5**
4. Expected Results **8**
References **8**

List of Figures

Figure 1. Framework of the system 4
Figure 2. Simulated environment and build snapshots 5
Figure 3. Sample assemblies in the repository 6
Figure 4. LEGO part meshes 6
Figure 5. Decomposition of the arch into a build sequence 6
Figure 6. Camera feeds and detections 7
Figure 7. Collision-sphere model of the two arms 7
Figure 8. Precision gap and wriggle primitive 7
Figure 9. Expected bracing, wriggling and finished build 8

List of Tables

Table 1. Simulation software and what is built in it 3
Table 2. Objectives: done and left to do 7

1. Introduction

1.1 Dual-arm LEGO assembly as a scaffolding problem

A partly built LEGO structure must carry the insertion press of every new brick, about 8.9 N per stud or 71 N for a 2x4, through stud joints that hold only about 11.3 N per stud in tension. Arches, corbels and bridges therefore fail during construction unless something holds them. A second arm can hold them, but this raises its own problems. The support must be in the right place: in our earlier simulation, two bracing rules differed by up to 100 percentage points at a single step. The two grippers work within millimetres of each other around a small structure, and early runs recorded arm-to-arm contact forces of up to 442 N. A brick that drops between two placed neighbours has only 0.1 mm of play per side, while the cell's aim error is about 0.8 mm (one standard deviation), so position control alone cannot place it. Finally, the 17.5 mm finger pad is wider than a 2-wide brick (15.8 mm) and grazes placed neighbours.

1.2 Objectives

This phase has three objectives, set by the course requirements.

- **Objective 1: take the target structure as direct input.** Converting an image into a target structure is not required. *Approach:* a structure is selected by name and read as a brick list on the LEGO grid (type, position, layer, yaw) from the repository's library of twelve assemblies; no image is needed.
- **Objective 2: automatically generate the assembly sequence and determine where temporary support is needed.** *Approach:* the planner orders the bricks bottom-up layer by layer, and a force-balance model of the stud joints flags the steps at which the partial structure would fail under the insertion press, together with the grasp arm A should use there.
- **Objective 3: use both robotic arms to autonomously assemble two different structures, each with at least eight LEGO bricks and at least one section requiring temporary support.** *Approach:* arm B places every brick while arm A holds the structure at the generated support steps; the two test structures are the arch (11 bricks, one supported step) and S3 (16 bricks, three supported steps).

2. Methodology

The system runs in a simulated dual-arm cell; Table 1 lists the software and Figure 2 shows the cell. Figure 1 shows the framework: the target is first decomposed into an ordered build plan with its support steps (a); then, for every brick, arm A braces when the plan asks for it (c) and arm B places the brick with a press or, at a tight socket, a wriggle (d), with the two arms coordinated as in (b).

Table 1. Simulation software and what is built in it.

Item	What we use
Simulator	NVIDIA Newton 1.2.1 (Warp + MuJoCo-Warp solver, GPU), run in the Isaac Sim 6.0 Python environment
BrickSim [1]?	Not used at run time. Used earlier to calibrate the stud-joint model and cross-check the planner
Built by us	Dual-arm FR3 cell, studded brick meshes, breakable stud joints, cameras and brick detectors, planner, bracing, executor
Wrapped / reused	cuRobo [3] (motion planning); StableLego [2] force balance (reimplemented); Isaac-Lab-style RL interface
Wriggle training	Separate MuJoCo 3.3.7 model with stud interference

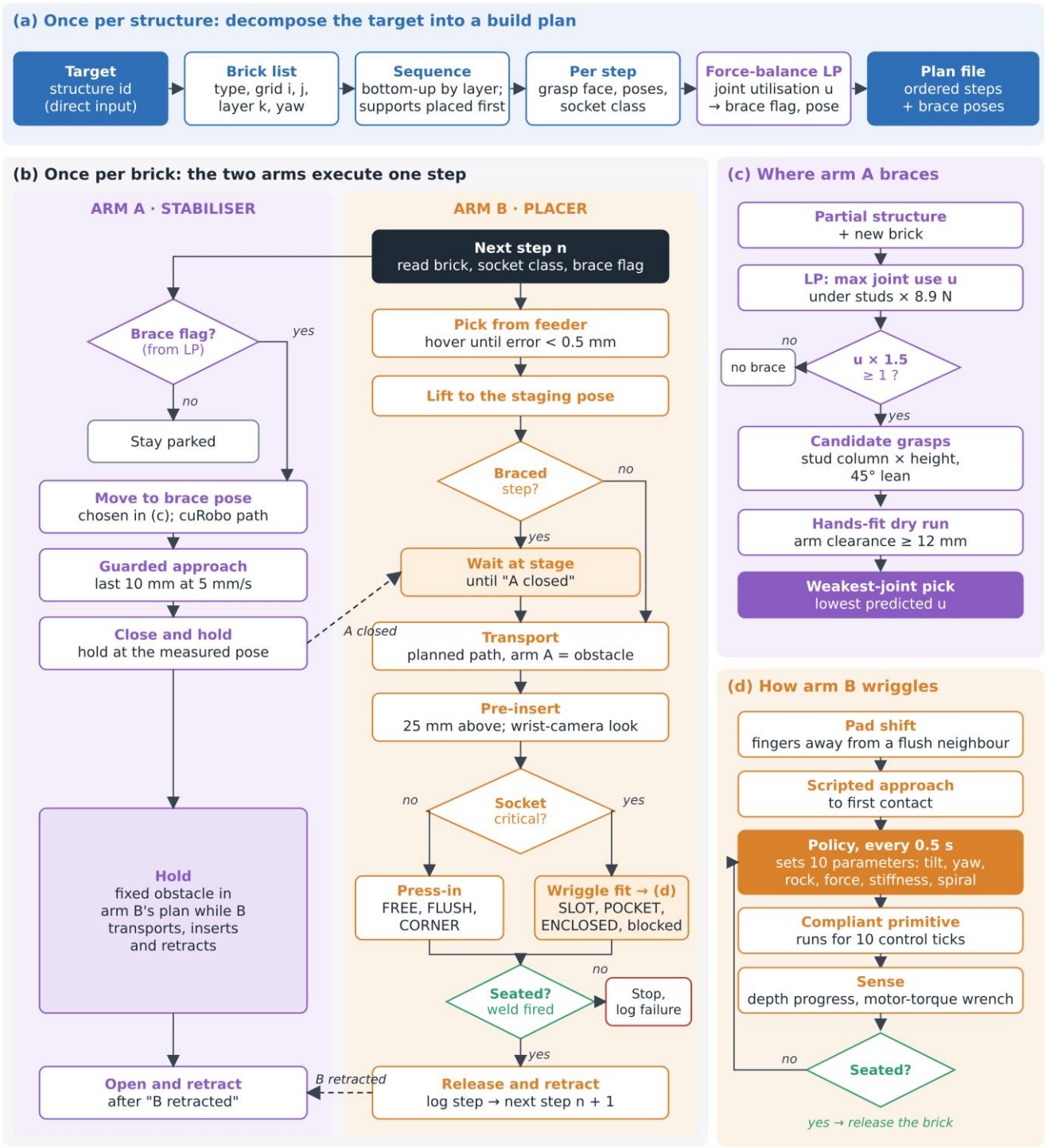


Figure 1. The framework. (a) Once per structure, the target is decomposed into a build plan. (b) Once per brick, the two arms execute one step; diamonds are decisions and dashed arrows are the events one arm waits for. (c) How arm A chooses where to brace. (d) How arm B wriggles a brick into a tight socket.

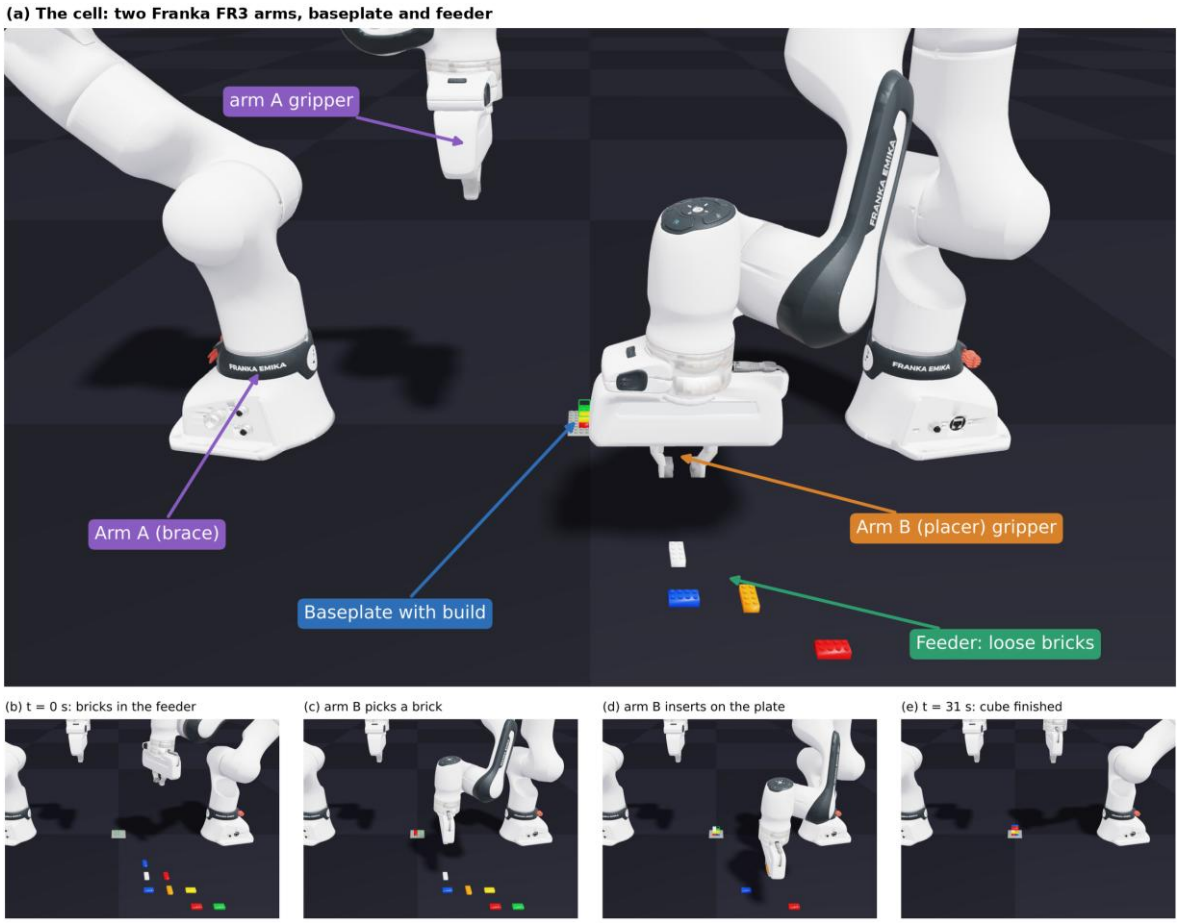


Figure 2. The simulated environment. (a) Arm A (bracing) on the left, arm B (placing) on the right, the baseplate between them and loose bricks in the feeder. (b)–(e) Snapshots from a simulated build of the cube.

3. Design Specification and Progress

The input is one of the twelve assemblies in Figure 3, selected by name and read as a list of bricks on the LEGO grid; no image is used. Every brick is one of the generated part meshes in Figure 4: hollow shells with studs and inner tubes, built in code for any size. The planner turns the brick list into a build sequence, as Figure 5 shows for the arch: the bricks are grouped into five layers, placed bottom-up in eleven steps, and the force model marks one step, the crown, as needing support, with arm A gripping the base of the left pier while arm B inserts it.

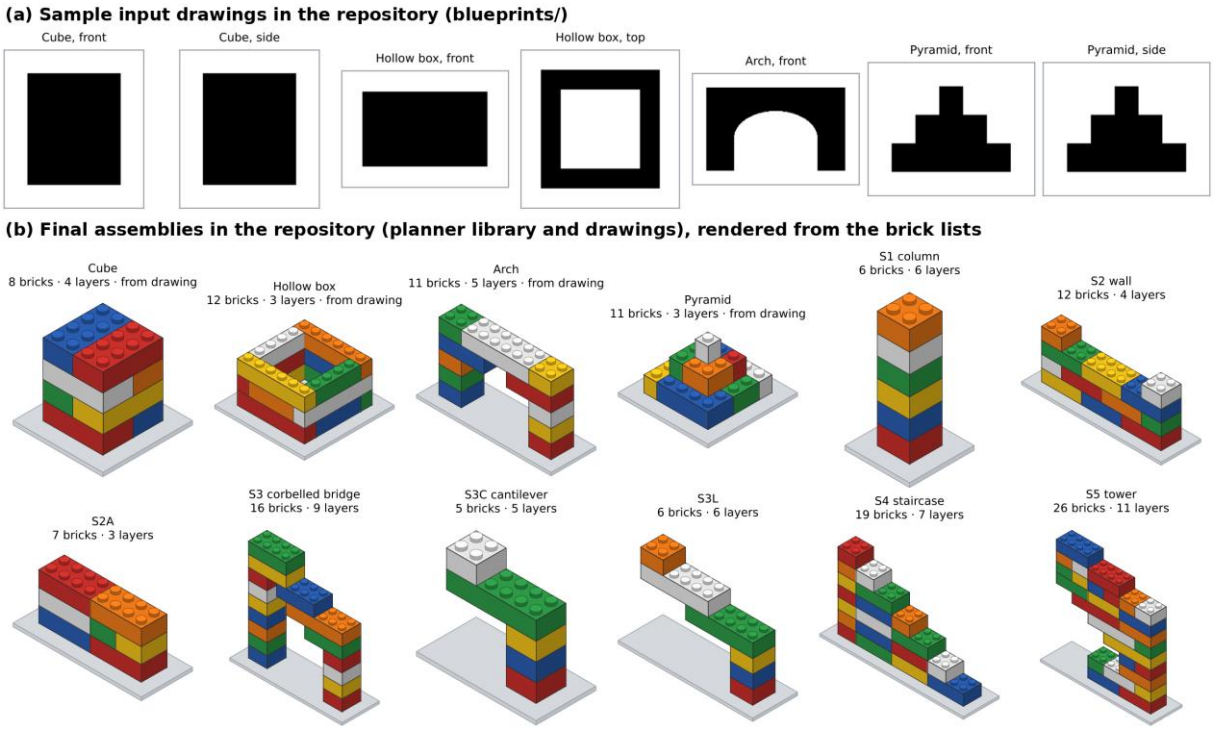


Figure 3. The assemblies we currently work with. (a) Drawings from which four of them were originally carved (not used at run time). (b) All twelve final assemblies, rendered from the brick lists that the planner reads; colours mark build order.

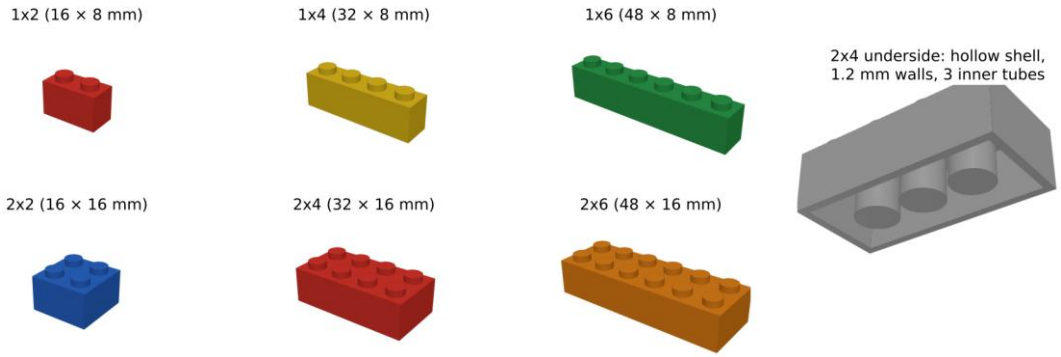


Figure 4. LEGO part meshes generated by the simulator, at a common scale. Each brick is 9.6 mm high plus 1.7 mm studs; the underside is a hollow shell with inner tubes.

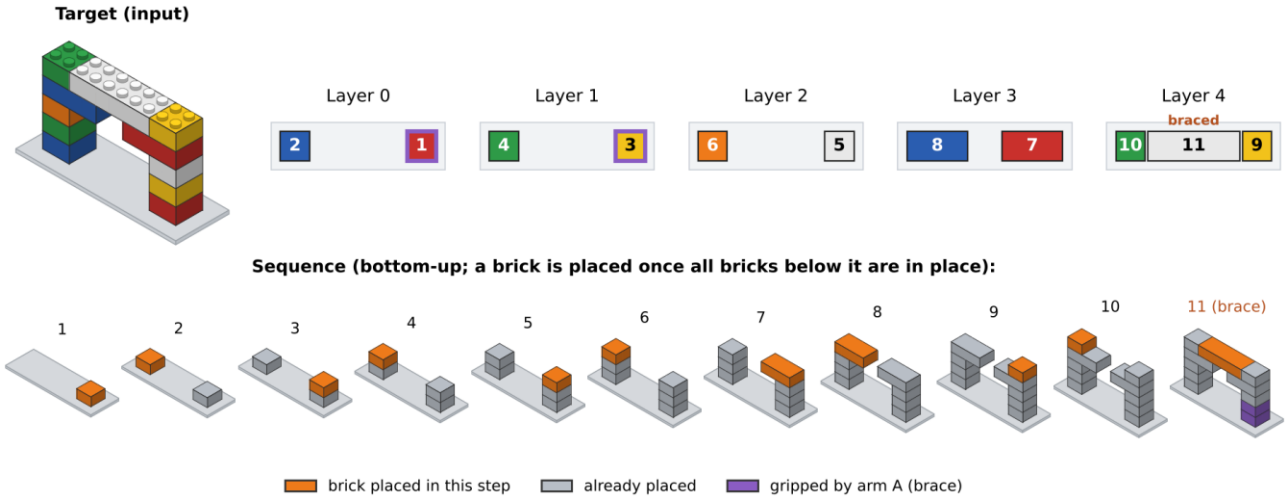


Figure 5. The arch decomposed into five layers (top-down maps; numbers are build steps) and the resulting 11-step sequence. Purple marks the bricks arm A grips while the crown is inserted.

In the cell, the bricks are found by simulated RGB-D cameras (Figure 6). The top camera reads every brick's colour, size, position, yaw and layer and the pose of the baseplate, and the wrist cameras look at the target studs just before insertion. Motion between the feeder and the build is planned with a sphere model of each arm (Figure 7), so that each arm avoids the other. Where a brick must go between two placed neighbours, the play is far smaller than the aim error, so arm B wriggles it in instead of aiming (Figure 8).

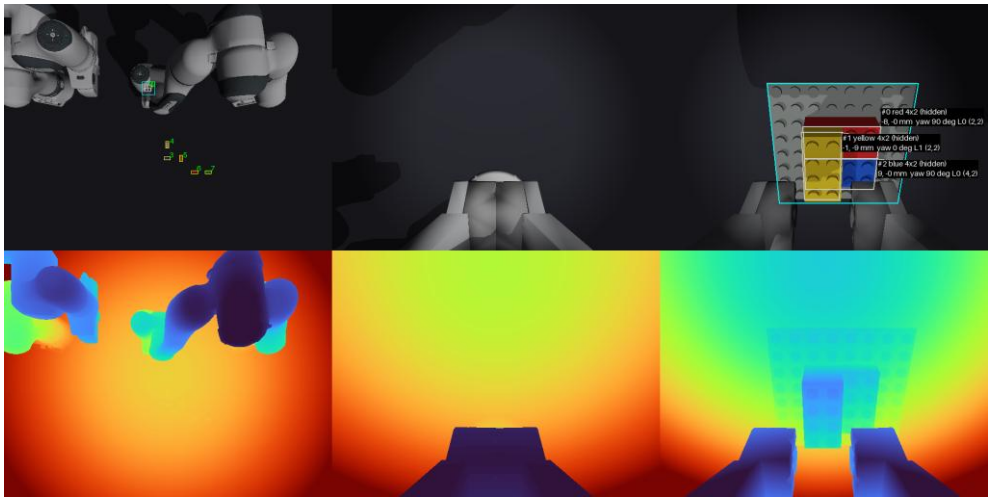


Figure 6. Camera feeds in the cell. Top row: RGB from the top camera (with detected bricks numbered), wrist_A and wrist_B (with each detected brick's colour, size, position, yaw and layer). Bottom row: the matching depth images.

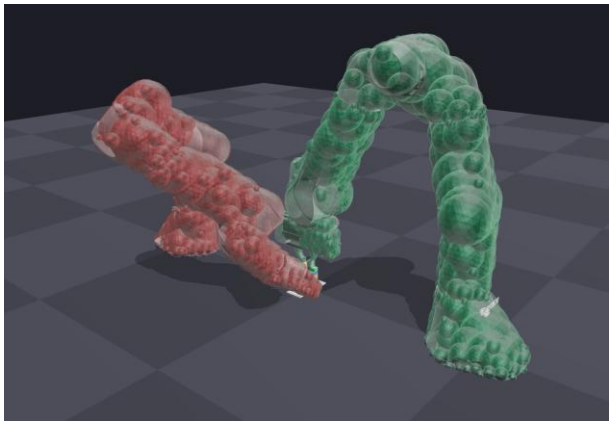


Figure 7. The collision model used by the motion planner: each FR3 is approximated by spheres fitted to its links (arm A red, arm B green), shown at the crown step of the arch.

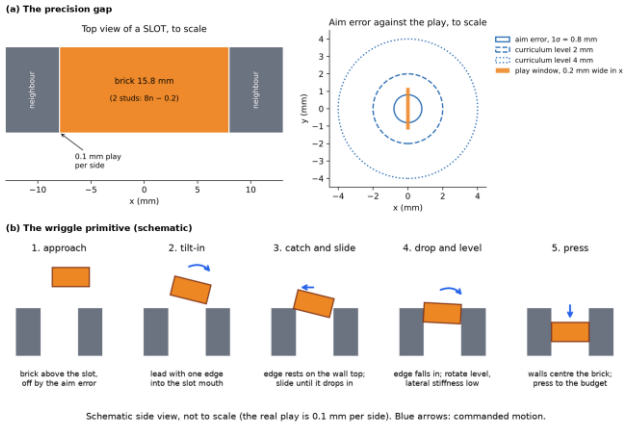


Figure 8. Wriggle fit. (a) A SLOT socket to scale, and the aim error against the 0.2 mm play window. (b) The five phases of the wriggle primitive.

Table 2 compares the three objectives with what has been done.

Table 2. Objectives: done and left to do.

Objective	Done	Left to do
1. Target structure as direct input	All twelve assemblies are selected by name; no image is used.	—
2. Automatic sequence and support	Order and support steps generated automatically: arch 1, S3 3.	Switch the runs from the default "no brace" to the weakest-joint strategy.
3. Two dual-arm builds of ≥ 8 bricks with support	Arch 11/11 and S3 16/16 built, 3/3 runs each, with arm A parked; two-arm motion without contact shown on the cube.	Builds with arm A holding every support; fix the arm-to-arm contact at braced steps.

4. Expected Results

Figure 9 shows the target outcome: arm A bracing the arch while arm B inserts the crown, the crown being wriggled into its slot, and S3 finished in the centre of the cell.

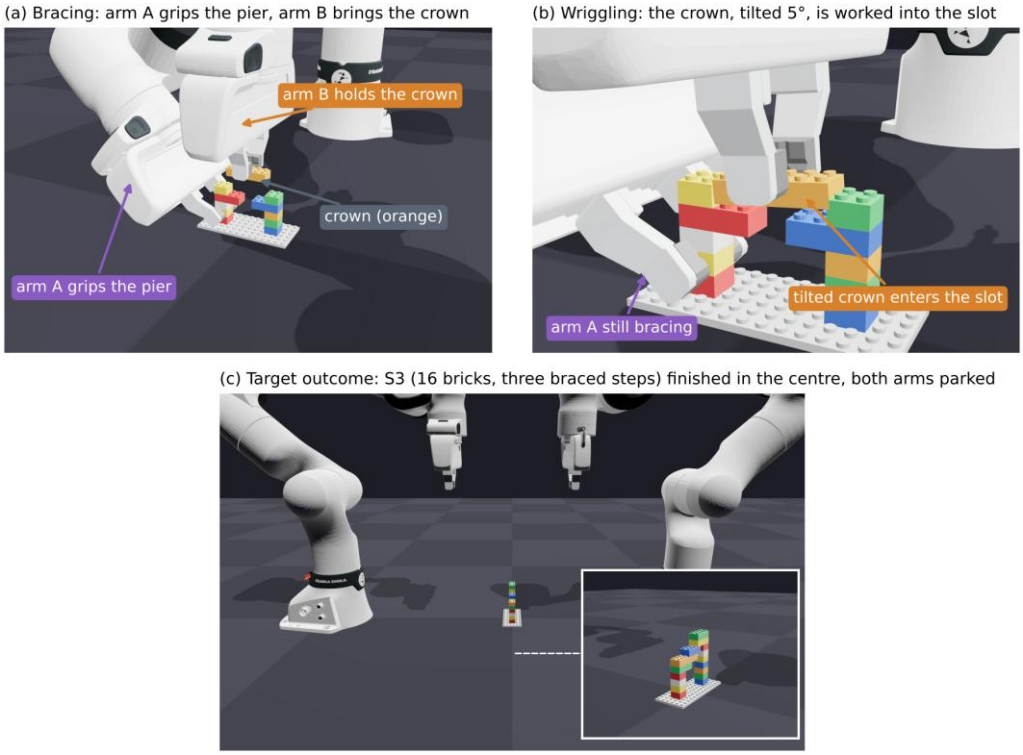


Figure 9. What the finished system should look like. (a) Bracing at the arch crown. (b) Wriggling the crown into its slot. (c) S3 completed in the centre of the cell. Renders of the cell posed from the plan with inverse kinematics, not simulation results.

References

[1] H. Wen et al., "BrickSim: A physics-based simulator for manipulating interlocking brick assemblies," arXiv:2603.16853, 2026.

[2] R. Liu, K. Deng, Z. Wang, C. Liu, "StableLego: Stability analysis of block stacking assembly," IEEE Robotics and Automation Letters, 9(11), 2024.

[3] B. Sundaralingam et al., "cuRobo: Parallelized collision-free robot motion generation," ICRA, 2023.